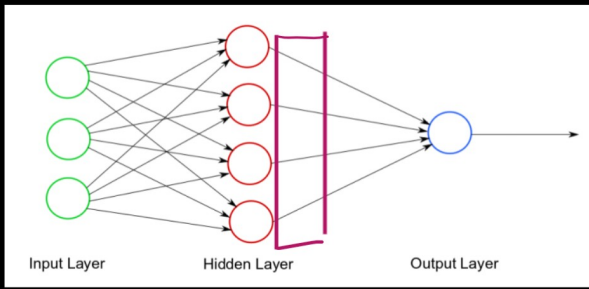
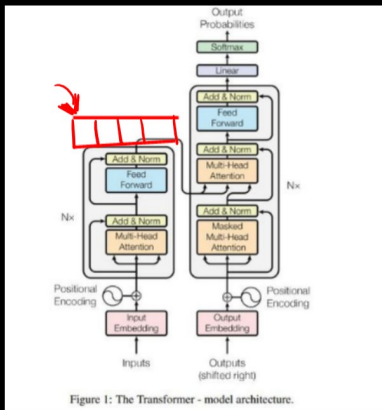


Transformer



← NN
 ← word2vec
 ← intermediate o/p of nn

machine translation



- ① text classification
- ② Summarization
- ③ translation
- ④ text generation

← NLP

SOTA

state of art

LLM (GPT, Gensai, hf)

AI

Embedding

intermediate o/p

Prompt

GPT

text classification

Pos/Neg
spam/ham

Summarization

translation (Eng → Hindi)

my name is anany

① Pretaining
(Next token)

Previous sent. → Next token

(transformer decoder)

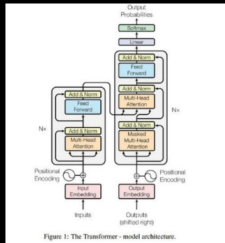
① 2017-19 → transformer → Machine translation

text classification =
Summarization =

② BERF → Google → MLM Sunny → is → [Mask] → mentor

masked

③ GPT → OpenAI → Autoregressive training Sunny is → is → mentor.
next token



→ LAM (text generation)

→ (embedding model)



convert our data into meaning numbers

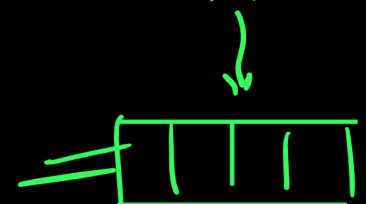
(Numbers)
embedding

- { 1 word }
- { 2 sent- }
- { 3 Para/document- }
- { 4 Img }
- { 5 Andro }
- { 6 video }

(transformer) (embedding)

self attention → FF

NN → word2vec (HL)



SOTA → transformer → SOTA embedding model → meaningful numbers
Any type of data

Feature	Encoding (One-hot / TF-IDF)	Neural Network Embedding (Word2Vec)	Transformer Embedding (OpenAI, Gemini, Sentence Transformer Based Model)	Reason
Meaning capture	No semantics	Some semantics	Deep contextual semantics	Encoding is frequency-based, NN learns patterns, Transformer understands context
Context awareness	None	Limited (window-based)	Full context (bidirectional)	Transformer processes the full sentence at once
Vector type	Sparse	Dense	Dense	Encoding has mostly zeros, NN/Transformer give compact representations
Same word embedding	Same vector always	Same vector always	Different per context	Transformer gives dynamic embeddings and handles multiple meanings
Performance	Weak	Good	Best	Transformer uses attention and is trained on large-scale data

How to select best embedding model

1. Embedding Quality (Most Important): Check benchmark performance:

Look at:

- MTEB leaderboard
- BEIR benchmark: <https://arxiv.org/abs/2104.08663>

2. Dimensionality

Example:

- 384 dim → lightweight
- 768 dim → balanced
- 1536 dim → high quality but heavy

Higher dimension:

- Better semantic representation
- But more storage cost
- More memory usage
- Speed impact

3. Cost

Close source (Open AI embedding):

- Easy, high quality, API cost

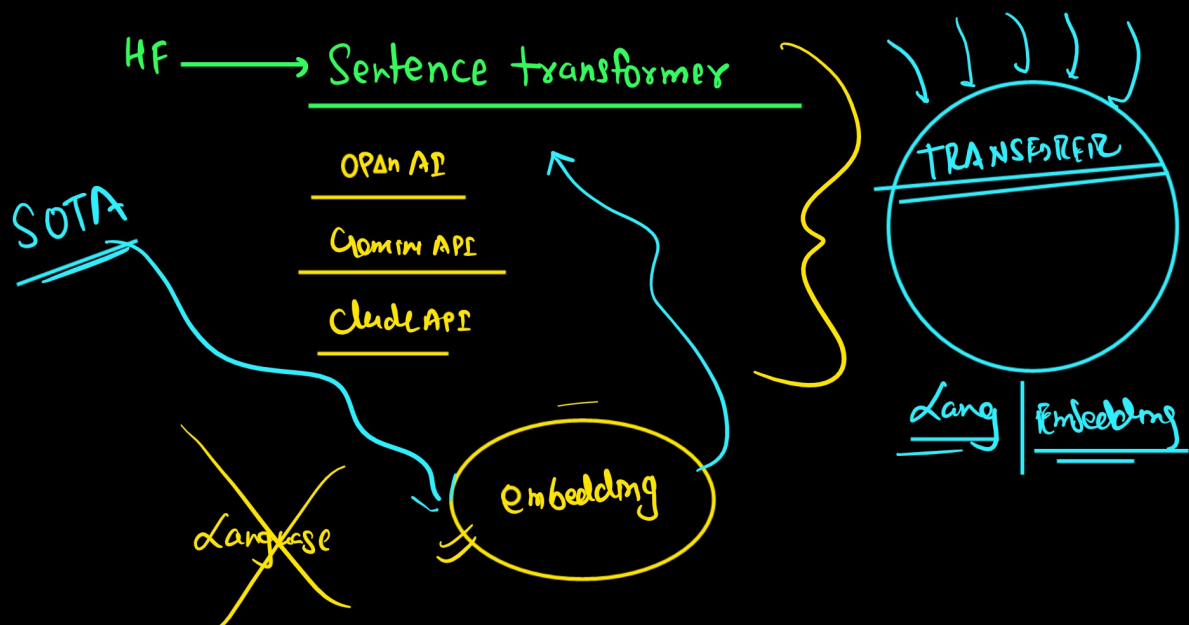
Open source:

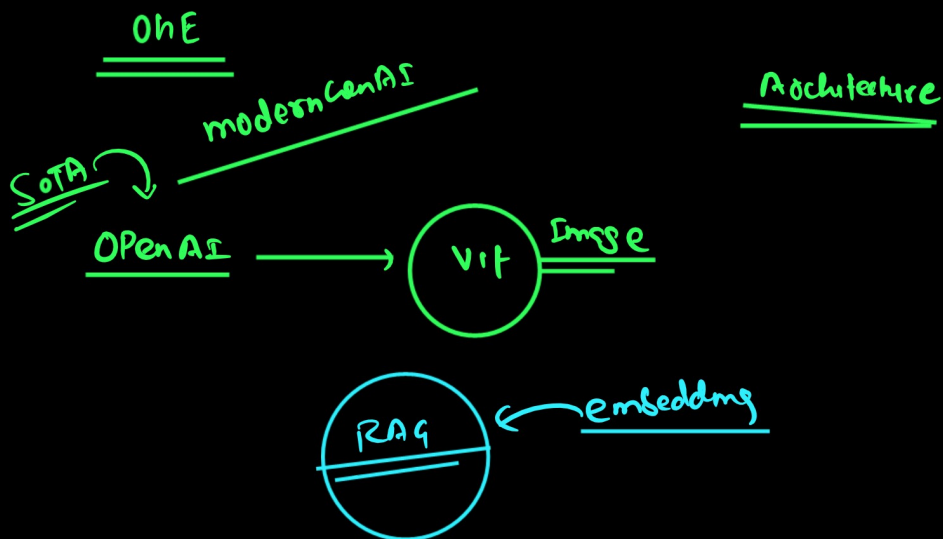
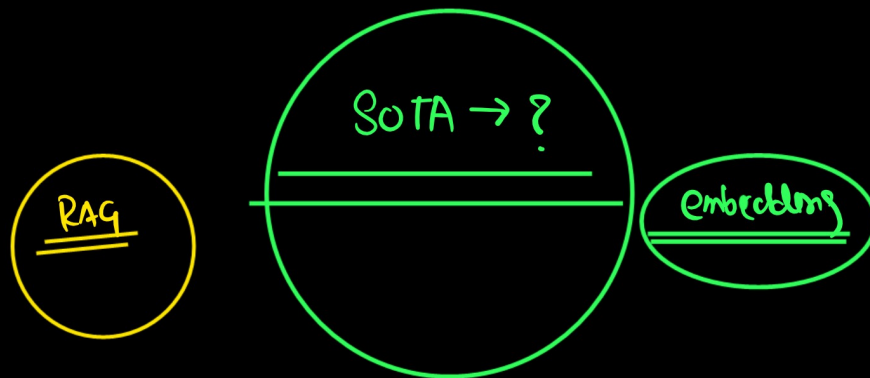
- Free but infra + scaling cost

4. Domain Suitability

General model:

- all-MiniLM
- OpenAI embedding
- Gemini embedding





① SOTA / embedding model ← text, image, audio, video

② Similarity Search

- ① Dot ✓
- ② COSINE SIM ✓
- ③ EUCLIDEAN DIS (L2) ✓

Which best one RAG chapter ←

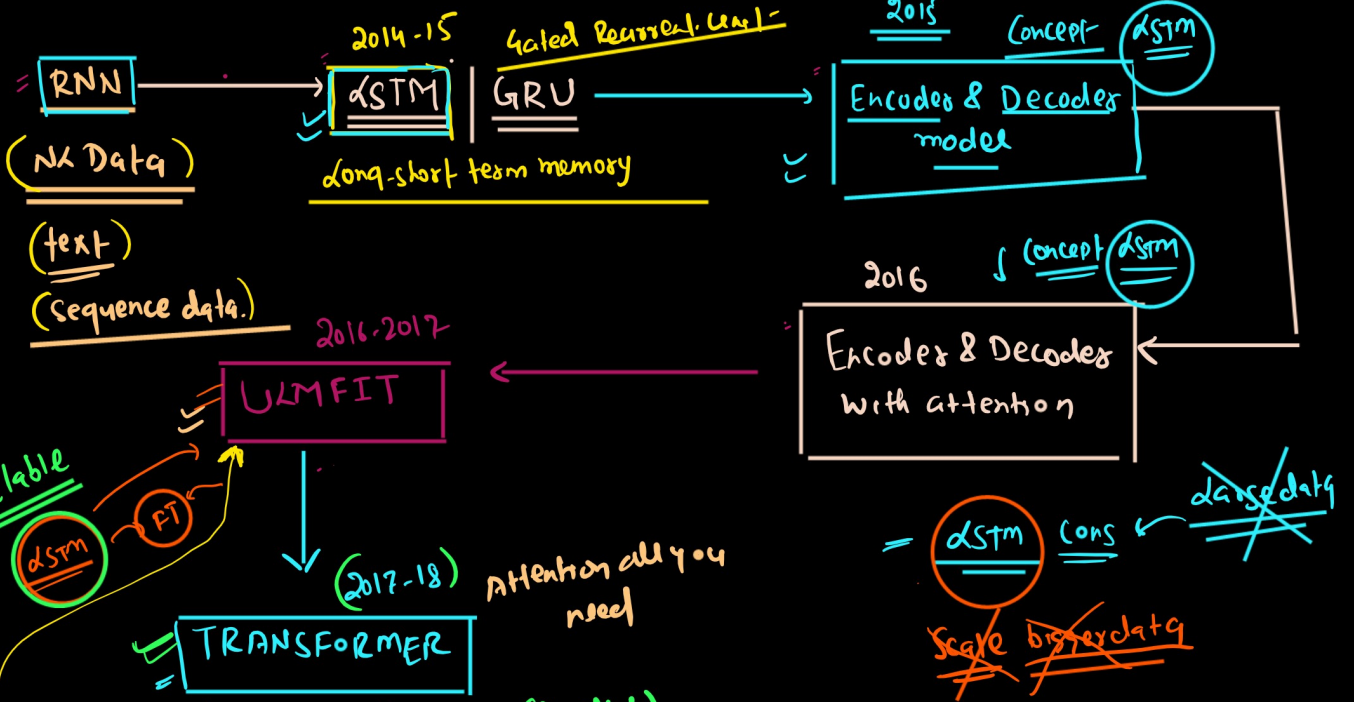
PT

③ evaluation of the embedding model

④ TRANSFORMER ← 3/4 clones

TRANSFORMER

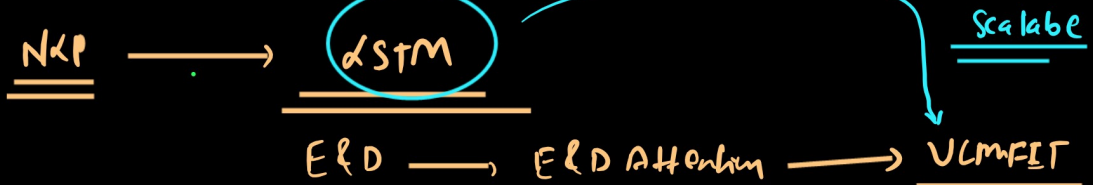
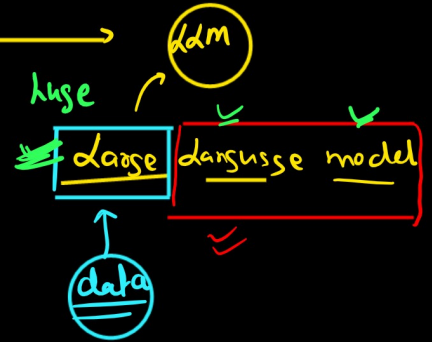
NLP —> ADVANCE NLP



dSTM v/s transformer

Lang model => (Next token Prediction)

Seq => AI Model



(transformer) dSTM ULMFIT

large language model

Lang => Early stage Lang
{BERT, GPT-1, 2, 3}

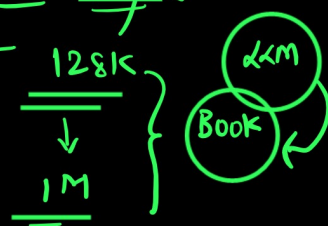
Interview 2024

Feature	RNN/LSTM	Transformer
Processing style	Sequential (step by step)	Parallel (all tokens at once)
Speed	Slow	Fast
Long-term dependency	Limited (vanishing gradient issue)	Very strong
Architecture	gates: input, forget, output	Attention-based (self-attention)

Parallel (dot's of data)

long sequence dsm (context)

4PT44.5.5.1



Attention / (self-attention)

transformer

Pre-req.

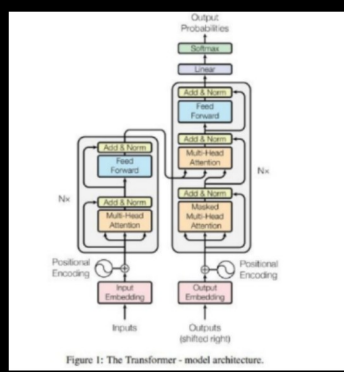
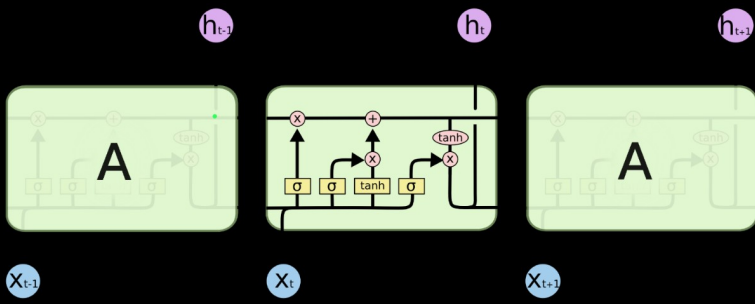
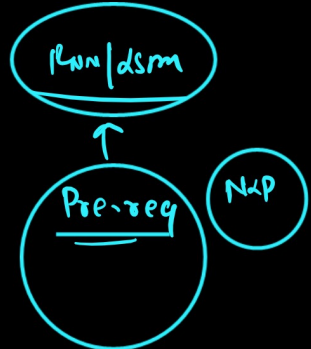
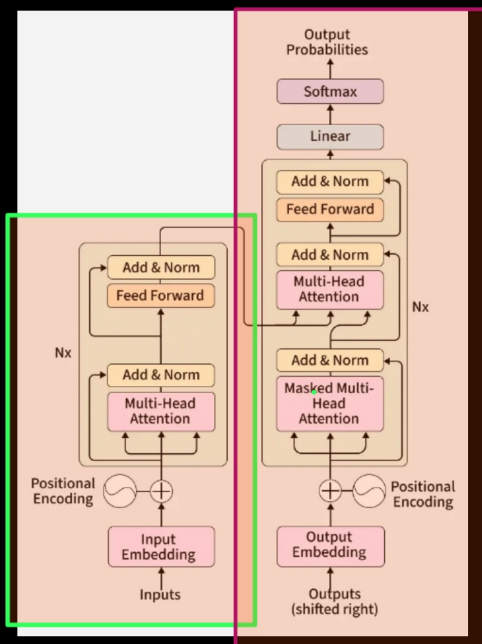


Figure 1: The Transformer - model architecture.

- 1 RNN → transformer
- 2 research paper
- 3 lstm v/s transformer





Encoder
BERT

Decoder
GPT

LLM

- 1 GPT
- 2 LLaMA
- 3 Mistral
- 4 Deepseek
- 5 Qwen
- 6 OpenAI / Anthropic / Google
- 7 Gemini

Decoder

Machine translation

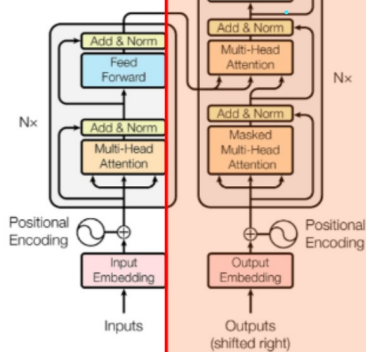
2018

ERD

Encoder →

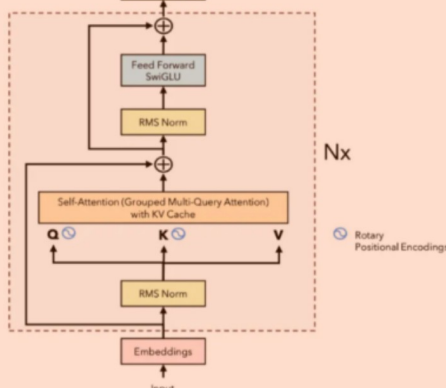
Decoder →

TRANSFORMERS ARCHITECTURE



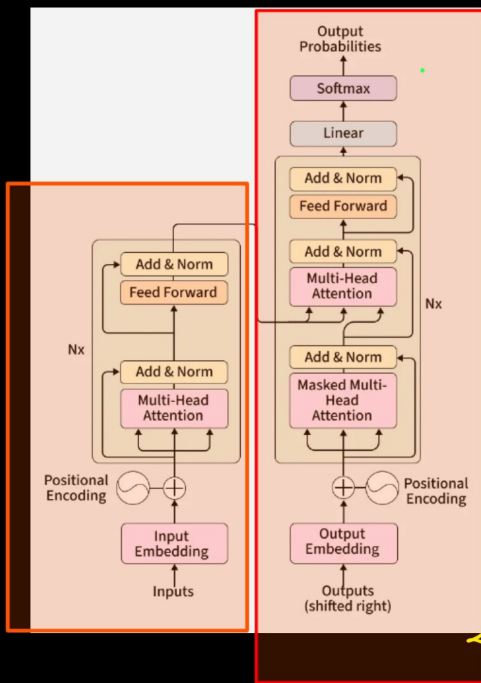
VS.

LLAMA ARCHITECTURE



hi I am sorry
↓
hi I am sorry

Zoumana K.



Encoder Side

1. Input sequence $\leftarrow$ Parallel (token)
2. Input embedding $\leftarrow$ word2vec
3. Positional encoding $\leftarrow$ Sequence of the data
4. Self-attention (Multi-head attention) $\leftarrow$ Q, K, V (Linear layers)
5. Add & Layer Normalization
6. Feed Forward Network (ANN)
7. Residual connection + Layer Normalization
8. Repeat encoder block N times (same process in each block)

6 stack

Decoder Side

1. Output sequence (shifted right)
2. Output sequence embedding (dimension ~512)
3. Positional encoding
4. Masked Multi-head Attention
5. Add & Layer Normalization
6. Residual connection
7. Cross Attention (encoder + decoder interaction)
8. Add & Layer Normalization
9. Feed Forward Network
10. Add & Layer Normalization
11. Linear layer
12. Softmax
13. Final output (probabilities)
14. Repeat decoder block multiple times

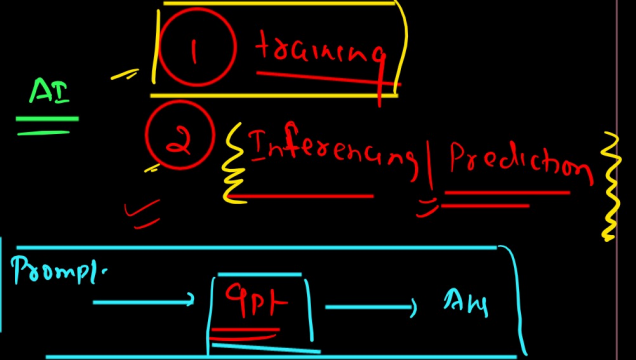
training

(OpenAI Dev)

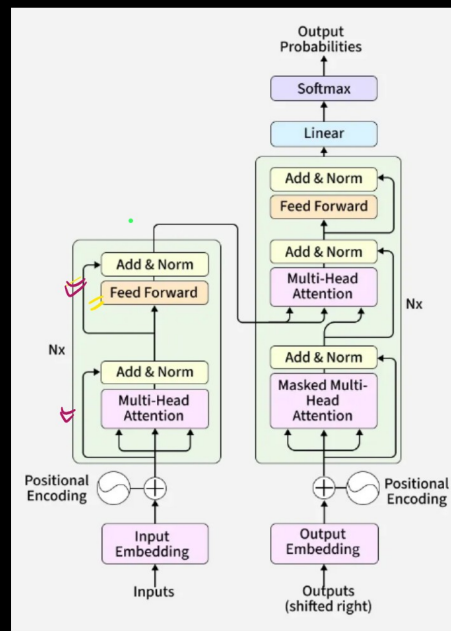
GPT

Provided

1. NN
2. LSTM
3. transformer



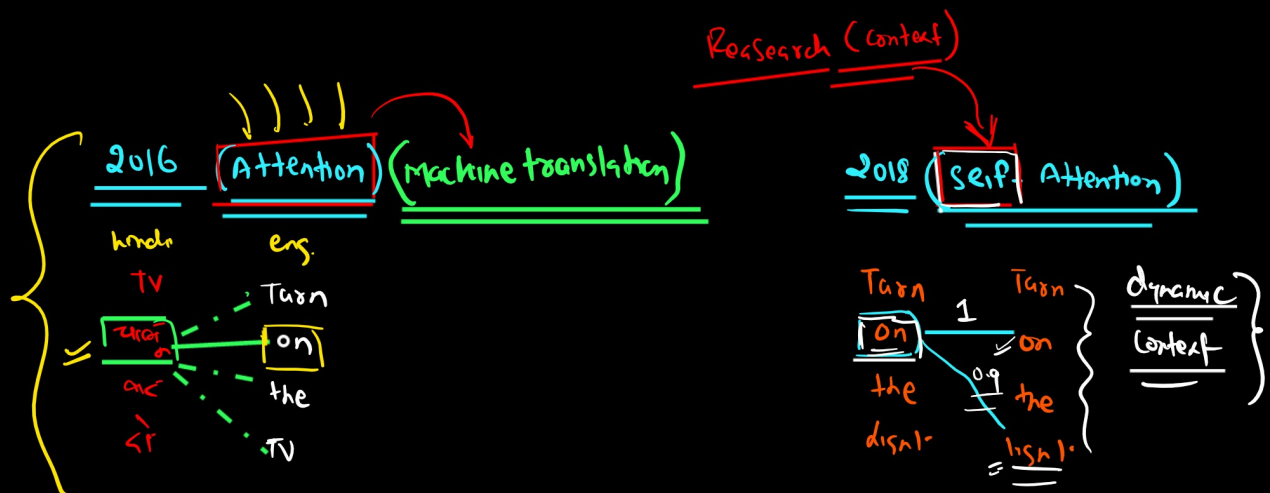
transformer $\rightarrow$ Jay Alammar



- ① Positional encoding
- ② Self Attention | multihead self attention
- ③ layers | Residual | FF
↑
NN

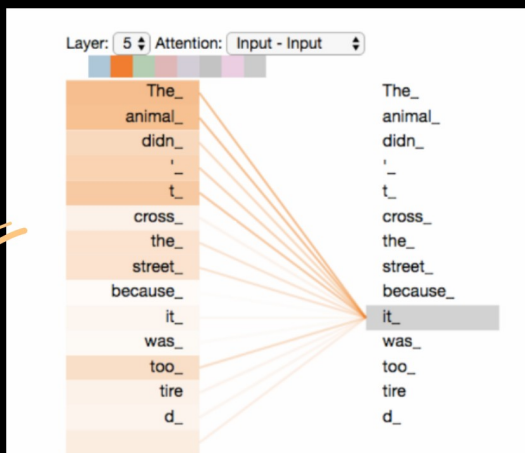
↑ ↑ ↑ ↑
[Sunny] [in] [a] [mentor]

- Data →
- ① tokenization
 - ② embeddings (word2vec)
 - ③ Positively
 - ④ PE
 - ⑤ Attention → self attention
 - ⑥ FPNN
- } learnings



It was trying to find out the weightage of any word in the same sentence

Self-attention = a word looking at other words in the same sentence to understand its meaning



Q, K, V
matrix

Self Attention → dynamic embedding

Apple launch a new phone while i was eating the Apple.

Company (under Apple)

Fruit (under Apple)

$S_1 \rightarrow$ money bank grows

$\rightarrow S_2$ River bank flows

bank $\rightarrow w_1 \times \text{money} + w_2 \times \text{bank} + w_3 \times \text{grows}$

Self Attention

bank $\rightarrow w_1 \times \text{River} + w_2 \times \text{bank} + w_3 \times \text{flows}$

Self Attention

money $\rightarrow w_1 \times \text{money} + w_2 \times \text{bank} + w_3 \times \text{grows}$

